TENET SYSTEMS

Internal Architecture Whitepaper

How the System Actually Works

v1.0 · 2026 · Confidential · Internal Distribution Only

For: Founders · Engineering hires · Technical investors · FI and LP design partners

1. What This Document Is

The Tenet Systems Technical Architecture Whitepaper describes what Tenet does and why it exists. This document describes how it does it — the internal mechanics, the order of operations, and the exact responsibilities of every component. These are two different documents for a reason. The external whitepaper answers the question an investor asks. This document answers the question an engineer or a technical partner asks.

This document is the authoritative internal reference for anyone building on, integrating with, or extending the Tenet architecture. It will eventually be merged into a single grand whitepaper. For now it stands alone.

ONE-LINE SUMMARY An intent enters. The Orchestration Engine conducts every engine to do its specific job. The Ledger records every step. No engine talks to another directly. Only the Orchestration Engine writes to the Ledger.

2. Architecture Principles

Every design decision in Tenet's architecture flows from four principles. These are not aspirational. They are structural constraints that every component must obey.

2.1 Engines are stateless pure workers

No engine holds state between calls. No engine remembers a previous order. When an engine is called, it receives everything it needs in that call, does exactly one job, and reports the result back to the Orchestration Engine. When it is done, it retains nothing. This means engines can be scaled horizontally without coordination — ten instances of the Execution Engine can run simultaneously without any of them needing to know about the others.

2.2 Only the Orchestration Engine writes to the Ledger

The Clearing Ledger is the single source of truth for every order in the system. No engine writes to it directly. The Execution Engine, Settlement Engine, and Reservation Engine all report their

results back to the Orchestration Engine via callbacks. The Orchestration Engine is the only component that reads and writes order state. This means the Ledger is always consistent — there is no race condition where two engines simultaneously update the same order record.

2.3 Engines never talk to each other

The Execution Engine does not call the Settlement Engine. The Settlement Engine does not call the Reservation Engine. The Reservation Engine does not call the Execution Engine. Every engine only ever communicates upward — back to the Orchestration Engine. The Orchestration Engine is the conductor. It coordinates every engine and sequences every step.

2.4 Vault balances are Tenet-internal, not on-chain

Tenet maintains its own internal view of every vault balance. That view is: actual custody balance minus active reservations. This is the balance Tenet uses for all routing and reservation decisions. The actual assets in Fireblocks or Copper never move until an order executes. Before that point, everything is reservation logic on Tenet's internal ledger. This is how the one-pool-of-cash model works in practice — the FI's $10M sits in one vault, and Tenet's internal accounting tracks exactly how much of it is spoken for at any moment.

3. The Components

Tenet's architecture has six components. Each has a single defined responsibility. None of them do anything outside that responsibility.

Component	Responsibility
CLI	The FI-facing interface. Accepts intent submission, displays quotes, handles confirmation, displays lifecycle animations. Calls ORCH.submitIntent() when an intent is confirmed.
Orchestration Engine (ORCH)	The conductor. Receives every intent. Sequences every engine call. Makes every routing decision. Writes every state transition to the Ledger. The only component that touches the Ledger.
Reservation Engine	Sub-component of ORCH. Owns all vault reservation logic. Two gates: Gate 1 locks FI vault balance, Gate 2 locks LP vault inventory. Releases reservations on completion or failure.
Execution Engine	Finds the best LP fill plan for a given intent. Delivers assets from LP vault(s) to the FI destination wallet. Reports back to ORCH via callbacks. Never touches the Ledger.
Settlement Engine	Extracts settlement from the FI vault into the Tenet Settlement Vault. Routes LP payout to their native chain on ORCH instruction. Reports back to ORCH via callbacks. Never touches the Ledger.
Clearing Ledger	The single source of truth. Records every order, every state transition, every engine event, every reservation. Only ORCH writes to it. Everything else reads from it.

4. The Vault Structure

Every asset in the Tenet system lives in one of four vault types. No assets flow outside this structure.

Vault	Owner	Purpose
FI Settlement Vault	FI + Tenet (2-of-2 multisig)	FI pre-funds this vault once. Tenet reserves against it per intent and extracts on delivery.
LP Liquidity Vault	LP + Tenet (2-of-2 multisig)	LP pre-positions assets here per chain. Tenet reserves inventory against it and co-signs transfers on order execution.
Aggregation Vault	Tenet (sole)	Ephemeral. Created per multi-LP order. Receives assets from multiple LP vaults, then makes one clean delivery to the FI. Destroyed when order completes.
Tenet Settlement Vault	Tenet (sole)	Receives extracted settlement from FI vault. Routes LP payout after delivery confirms. Tenet fee accumulates here permanently.

4.1 Internal Vault Balances

Tenet maintains an internal balance for every vault. This is separate from the actual custody balance held in Fireblocks or Copper. The internal balance is:

```
internal_balance = actual_custody_balance - active_reservations
```

When a reservation is posted, the internal balance decreases. The actual Fireblocks vault is untouched. When an order executes and assets actually move, both the internal balance and the custody balance decrease. When a reservation is released (order failed or completed), the internal balance restores.

This distinction is critical. Two concurrent orders from the same FI both check the internal balance — not the custody balance — so neither order can be accepted if the combined reservation would exceed what is actually available. The reservation system is entirely internal to Tenet and requires no on-chain locking.

5. The Order Lifecycle — Complete Walkthrough

The following describes every step of an order from intent submission to completed settlement. This is the canonical sequence. Every component, every decision, every state transition is described in order.

STATE MACHINE An order moves through exactly these states: LIVE → DELIVERING →

AWAITING SETTLEMENT → COMPLETED. The only exception states are INSUFFICIENT_BALANCE (order never becomes LIVE) and FAILED (order accepted but something prevented completion).

5.1 Intent Arrives at ORCH

The FI submits an intent from the CLI. The CLI calls ORCH.submitIntent(). ORCH receives the full intent object:

```
intent = {
  tenetAddress:   "GoldmanSachs_x800",   // FI identifier
  asset:          "ETH",                  // asset to buy/sell/swap
  usdcAmount:     500000,                 // USDC to spend (always USDC)
  deliveryChain:  "SOL",                  // chain FI wants asset delivered on
  settlementChain:"ETH",                  // chain FI vault lives on
  settlementAsset:"USDC",                 // asset in FI vault
  slipTolerance:  50,                     // bps. FI-specified.
  intent:         "buy",
  deadline:       null,                   // optional FI extension
}
```

ORCH creates an order object in ORDER_STORE with a unique OrderID. Status is PENDING. Nothing has been written to the Ledger yet.

5.2 Gate 1 — FI Vault Reservation

Before anything else, ORCH calls the Reservation Engine with the FI's Tenet address and the full settlement amount — usdcAmount plus Tenet's fee.

The Reservation Engine looks up the FI vault from the FI registry using the Tenet address. It checks the FI vault's internal balance — actual custody balance minus any existing reservations from other active orders. If the internal balance is sufficient, it posts a reservation for the full amount. The internal balance decreases immediately. The actual Fireblocks vault does not move.

```
Gate 1 check:
  FI vault internal balance = $10,000,000
  Existing reservations     = $1,200,000  (from two other active orders)
  Available                 = $8,800,000
  This intent requires      = $500,500    (usdcAmount + fee)

  $8,800,000 > $500,500  →  reservation posted ✓
  Available now becomes     = $8,299,500

If available < required:
  Status = INSUFFICIENT_BALANCE
  Order never becomes LIVE.
  This is not FAILED. The FI does not have enough available right now.
```

```
  FAILED means something went wrong during execution.
  INSUFFICIENT_BALANCE means the math does not work.
```

> INSUFFICIENT_BALANCE vs FAILED: These are two completely different outcomes and must never be conflated. INSUFFICIENT_BALANCE is a pre-execution check — the order is declined before it ever becomes LIVE. FAILED means the order was accepted, became LIVE, but something prevented it from completing — for example, slippage too tight for the market session, or a vault co-signing failure. The Ledger records both with different reason codes.

5.3 Order is Immediately LIVE

The moment Gate 1 succeeds, ORCH writes the order to the Ledger with status LIVE. The order appears on the LIVE sheet of the Clearing Ledger immediately. The Execution Engine has not been called yet. Gate 2 has not happened yet. The order is LIVE as soon as the FI vault is confirmed and reserved.

This is an important design point. LIVE does not mean the fill plan is ready. It means the FI's funds are confirmed available and reserved. Work is now happening on behalf of this order.

5.4 While LIVE — Execution Engine Finds the Fill Plan

With the order LIVE, ORCH calls the Execution Engine. It passes exactly what the intent contained — asset, usdcAmount, delivery chain, slippage tolerance. The spot price at this moment determines the quantity. The Execution Engine does not derive or calculate the quantity — the spot price does. The Execution Engine uses that quantity to search for LPs that can fill it.

The Execution Engine queries every LP vault registered for this asset on this delivery chain. It checks real-time available balances — no cache. It filters out any LP whose fill price exceeds the slippage tolerance. It runs Pareto elimination to remove dominated options. It ranks the survivors by blended cost. It builds a fill plan up to five LP legs. It reports the fill plan back to ORCH.

If no valid fill plan exists — because slippage is too tight for the current market session — the Execution Engine reports this to ORCH. ORCH releases the Gate 1 reservation, writes FAILED to the Ledger with reason SLIPPAGE_TOO_TIGHT, and the order ends. The FI's available balance restores.

5.5 Gate 2 — LP Vault Reservation

With the fill plan confirmed, ORCH calls the Reservation Engine again. This time it is locking LP inventory. Spot price may have moved slightly since the fill plan was built. Whatever the current quantity is at this exact moment — say 298 ETH instead of the 300 ETH calculated earlier — that is the quantity that gets reserved.

```
Gate 2 example:
  LP vault (Wintermute_SOL) actual balance  = 1,000 ETH
```

```
  Existing reservations on this vault       = 150 ETH  (from other active
orders)
  Available                                 = 850 ETH
  This order requires from Wintermute_SOL   = 298 ETH

  850 > 298  →  reservation posted ✓
  Wintermute_SOL internal balance now       = 552 ETH
  Actual Fireblocks vault still shows       = 1,000 ETH  (unchanged)

If any LP leg cannot be reserved:
  All partial LP reservations roll back.
  Gate 1 FI reservation releases.
  Order → FAILED.
  All internal balances restore.
```

5.6 LIVE → DELIVERING

Both gates passed. Fill plan locked. LP inventory reserved. ORCH now moves the order from LIVE to DELIVERING and writes this to the Ledger. The order disappears from the LIVE sheet and appears on the DELIVERING sheet.

Two things now fire simultaneously — this is the most important moment in the order lifecycle.

> Orchestrated atomicity: ORCH fires delivery and extraction at the same time using Promise.all(). Neither leg waits for the other. This eliminates settlement gap — the window between the FI receiving their asset and Tenet extracting the corresponding payment. Both happen in the same execution window.

5.7 Delivering — Execution Engine Delivers the Asset

ORCH gives the Execution Engine the LP Tenet address (or addresses for a multi-LP split) and instructs it to co-sign and transfer the asset to the FI's destination wallet.

For a single LP order: the Execution Engine co-signs a transfer from the LP vault directly to the FI destination wallet. One transaction.

For a multi-LP order: the Execution Engine co-signs transfers from each LP vault into the Aggregation Vault simultaneously. Once all LP legs are confirmed in the Aggregation Vault, a single transfer goes from the Aggregation Vault to the FI destination wallet. The FI always receives one clean transaction regardless of how many LPs were involved. The Aggregation Vault is then closed and zeroed.

The Execution Engine reports back to ORCH: asset delivered, here is the on-chain transaction hash as proof.

5.8 Delivering — Settlement Engine Extracts from FI Vault

Simultaneously with delivery, ORCH gives the Settlement Engine the FI's Tenet address and the full extraction amount — usdcAmount plus Tenet's fee. The Settlement Engine uses the

Tenet address to look up the FI vault internally, gets the co-signing key, and extracts the full amount from the FI vault into the Tenet Settlement Vault.

The Settlement Engine reports back to ORCH: extraction confirmed, funds are in the Tenet Settlement Vault, awaiting further instruction.

```
Extraction example:
  usdcAmount  = $500,000
  Tenet fee   = $500  (10bps)
  Total extracted from GoldmanSettlementVault  = $500,500
  TenetSettlementVault receives                = $500,500

  GoldmanSettlementVault internal balance:
    Before: $8,299,500 available (after Gate 1 reservation)
    After:  actual custody balance reduces by $500,500
            Gate 1 reservation released (no longer needed — funds moved)
```

5.9 DELIVERING → AWAITING SETTLEMENT

ORCH is waiting for two callbacks: delivery confirmed from the Execution Engine and extraction confirmed from the Settlement Engine. Only when both have reported back does ORCH advance the order.

ORCH transitions the order from DELIVERING to AWAITING SETTLEMENT and writes this to the Ledger. From the FI's perspective the transaction is complete — they have their asset. The rest of the process is invisible to them.

5.10 Awaiting Settlement — ORCH Makes the Routing Decision

ORCH now looks at the original intent and makes the routing decision for LP payout. It already knows everything it needs — the delivery chain, the settlement chain, the settlement asset, and the LP's native chain from the fill plan. No new information is needed.

Condition	Routing method	Who executes
Delivery chain = LP native chain, same asset	Direct transfer — no cross-chain movement needed	Settlement Engine
Different chains, settlement asset is USDC	CCTP — burn USDC on source chain, mint on destination chain	Settlement Engine
Different chains, settlement asset is not USDC	Everclear — cross-chain clearing for non-USDC assets	Settlement Engine
LP wants to receive a different asset than what Tenet holds	Internal swap — Execution Engine finds an LP willing to swap	Execution Engine reports back, Settlement Engine executes

ORCH passes this routing instruction to the Settlement Engine — or to the Execution Engine for internal swap — with everything fully resolved. The engines receive instructions, not decisions to make.

5.11 Awaiting Settlement — Routing Executes

The Settlement Engine executes the routing instruction. For CCTP: it burns USDC on the source chain, polls for Circle's attestation confirming the burn, then triggers the mint on the destination chain, and polls for mint confirmation. Once confirmed it reports back to ORCH. For Everclear: it posts the transfer intent to Everclear and polls for delivery confirmation. Once confirmed it reports back to ORCH.

For internal swap: the Execution Engine searches for an LP willing to provide the required asset in exchange for the USDC that Tenet holds in the settlement vault. Once found and executed, it reports back to ORCH. The Settlement Engine then handles the final delivery to the LP.

Throughout this entire process the order stays on the AWAITING SETTLEMENT sheet of the Ledger. ORCH writes every routing event to the Ledger as it happens.

> Routing failure: If CCTP and Everclear are both unavailable, the funds do not move. They remain in the Tenet Settlement Vault — safe, accounted for, waiting. The order stays on the AWAITING SETTLEMENT sheet. A retry scheduler activates. The LP will be paid when routing resumes. Funds never disappear. This is a contractual guarantee to every LP.

5.12 LP Payout

Once routing is complete — or if no routing was needed because chains are the same — ORCH gives the Settlement Engine the LP's Tenet address and the payout amount.

For a multi-LP split order, ORCH gives the Settlement Engine each LP's Tenet address and their individual share. Each LP's share is calculated based on their proportional contribution to the total asset quantity delivered. If Cumberland filled 27% of the order, Cumberland receives 27% of the net payout. The Settlement Engine executes a separate transfer to each LP on their respective native chains.

```
Multi-LP payout example:
  Net to LPs = $500,000 - $500 fee = $499,500
  Total qty  = 298 ETH

  Cumberland_SOL  filled 81 ETH  (27.2%)  →  receives $135,864
  Wintermute_SOL  filled 114 ETH (38.3%)  →  receives $191,409
  Galaxy_SOL      filled 103 ETH (34.5%)  →  receives $172,228

  Each LP receives payment on their native chain (SOL in this case).
  Each transfer uses the routing method ORCH determined.
```

5.13 AWAITING SETTLEMENT → COMPLETED

The Settlement Engine reports to ORCH that all LP payouts are confirmed. ORCH releases the Gate 2 LP vault reservations — the reserved inventory is freed, internal LP vault balances restore. ORCH transitions the order from AWAITING SETTLEMENT to COMPLETED and writes the final state to the Ledger.

The order moves to the COMPLETED sheet of the Clearing Ledger. It is immutable from this point. Every event, every engine callback, every transaction hash, every reservation and release is permanently recorded. This record is the foundation of Tenet's audit trail, reporting layer, and dispute resolution capability.

6. The Reservation System

The reservation system is how Tenet prevents race conditions. Without it, two concurrent orders from the same FI could both be accepted when the vault only has enough for one. Without it, two concurrent orders could both be routed to the same LP vault when it only has inventory for one. The reservation system solves both problems entirely within Tenet's internal ledger — no on-chain locking required.

6.1 How reservations work

Every vault has two balances in Tenet's system: the actual custody balance (what Fireblocks shows) and the internal available balance (custody balance minus active reservations). All routing and acceptance decisions use the internal available balance, never the custody balance directly.

A reservation is a record that says: "X units of this vault's balance are spoken for by order Y." The record is created when a gate passes and destroyed when the order completes or fails. While the reservation exists, no other order can count those units as available.

6.2 Gate 1 — FI vault

Posted immediately when an intent is accepted. Amount reserved: usdcAmount + fee. Released when the order is COMPLETED (because the actual funds have moved) or when the order fails at any point. If the order fails after extraction has already occurred, a reversal is triggered — the extracted funds are returned to the FI vault, and then the reservation is released.

6.3 Gate 2 — LP vault(s)

Posted after the Execution Engine confirms the fill plan, while the order is still LIVE. Quantity reserved: the asset quantity locked at this exact moment based on spot price. Released when the order is COMPLETED or FAILED. If the order fails after delivery has already occurred, this is a critical failure path that requires careful handling — see the failure handling section.

6.4 Why this makes the system safe for concurrent orders

Ten orders from ten different FIs can all be accepted simultaneously. Each one posts its own Gate 1 reservation against its respective FI vault. Each one posts its own Gate 2 reservations against LP vaults. The internal available balance of every vault is always current because every reservation and release updates it immediately. The system is consistent by design, not by luck.

7. The Clearing Ledger

The Clearing Ledger is not a database table. It is the operational backbone of the entire system. It is where every order lives, where every engine event is recorded, and where every state transition is written. It is the only place any of this is stored. ORCH is the only component that writes to it.

7.1 Ledger partitions

The Ledger is organised into sheets — partitions that hold orders currently in each state. An order exists on exactly one sheet at any moment. When ORCH transitions an order, it moves from one sheet to another.

Sheet	What it contains
LIVE	Orders where Gate 1 has passed, funds are reserved, and the Execution Engine is working on the fill plan.
DELIVERING	Orders where both gates have passed and asset delivery + settlement extraction are in progress simultaneously.
AWAITING SETTLEMENT	Orders where the FI has received their asset and Tenet holds the settlement funds. LP payout routing is in progress.
COMPLETED	Fully settled orders. Immutable. Every detail permanently recorded.
FAILED	Orders that were accepted but could not complete. Reason code attached to every failure.
RECONCILIATION	Unmatched vault deposits. Assets held safely pending investigation.
FI RESERVATIONS	Live view of all active Gate 1 reservations. Which FI vault, which order, how much locked, time remaining.
LP RESERVATIONS	Live view of all active Gate 2 reservations. Which LP vault, which order, how much inventory locked.

7.2 The reservation sheets never get cleaned up

The FI RESERVATIONS and LP RESERVATIONS sheets record every reservation that was ever posted, including released ones. This is intentional. A complete reservation history — every lock and every release with timestamps — is essential for audit, dispute resolution, and regulatory compliance. These sheets grow over time and are never purged.

7.3 The COMPLETED sheet is permanent

Every completed order stays on the COMPLETED sheet forever. It is append-only. Nothing is ever deleted. This is Tenet's proprietary data asset — a complete record of every institutional digital asset transaction that ever flowed through the system. Its value compounds with every order.

8. The Engine Event Log

Every meaningful action an engine takes is logged as an event. Events carry a timestamp, a source tag (ORCH, EXEC, RESV, SETT), a message, and a classification. These events are written to the Ledger by ORCH as it receives callbacks from each engine.

When you open any order in the Clearing Ledger and look at its engine events, you see a complete timestamped record of every decision: which LPs were considered, which were eliminated by Pareto and why, which LP vault reservations were posted, when delivery started, when extraction confirmed, which routing method was selected, when LP payout confirmed. Nothing is hidden.

This event trail is not optional. It is what makes Tenet's reporting layer possible, what makes disputes resolvable, and what makes the system auditable by regulators. Every engine is required to log every material decision.

9. Failure States

Every failure in Tenet's lifecycle has a defined response. Nothing hangs. Every exception produces a reason code, a Ledger entry, and a resolution path.

Failure	When it occurs	Response
INSUFFICIENT_BALANCE	Gate 1 check fails — FI does not have enough available in their vault	Order declined before becoming LIVE. Not a failure — a pre-execution check. FI notified with available balance and shortfall amount.
SLIPPAGE_TOO_TIGHT	Execution Engine cannot build a valid fill plan within the FI's tolerance	Gate 1 reservation released. Order → FAILED. FI told the tightest available price on the chain so they know what tolerance would work.
LP_LIQUIDITY_UNAVAILABLE	No LP inventory exists for this asset on this chain at any tolerance	Gate 1 reservation released. Order → FAILED. FI notified.
GATE_2_FAILURE	An LP vault reservation cannot be posted (insufficient available inventory)	All partial Gate 2 reservations roll back. Gate 1 releases. Order → FAILED.
VAULT_CO-SIGN_FAILURE	Custody API	Retry up to 3x with exponential

	unavailable during delivery or extraction	backoff. If all retries fail, order → FAILED, all reservations released.
CHAIN_TIMEOUT	Order deadline exceeded before delivery confirms	Reservation Engine fires timeout. All reservations released. Order → FAILED.
SETTLEMENT_ROUTING_FAILURE	CCTP and Everclear both unavailable for LP payout	Funds stay in Tenet Settlement Vault — never lost. Order stays AWAITING SETTLEMENT. Retry scheduler activates. Interest accrues per LP agreement if delay exceeds guarantee window.
EXTRACTION_CONFIRMS_DELIVERY_FAILS	Settlement extracted but asset delivery failed	Critical path. Extraction must be reversed immediately — full amount returned to FI vault. Logged to Ledger. Manual escalation if reversal fails.
RECONCILIATION	Unmatched vault deposit	Assets held safely in reconciliation. Automated matching attempted. Manual review after 1 hour if unresolved.

10. The Complete Flow in One View

The following is the entire order lifecycle from intent to completion, condensed into a single readable sequence.

```
FI submits intent via CLI
  │
  └─▶ ORCH receives intent, creates order in ORDER_STORE
        │
        └─▶ Reservation Engine — Gate 1
              Check: FI internal balance ≥ (usdcAmount + fee)?
              YES → reserve amount. FI internal balance decreases.
              NO  → INSUFFICIENT_BALANCE. Order ends here.
        │
        └─▶ ORCH writes LIVE to Ledger  ←── order is now LIVE
        │
        └─▶ Execution Engine — findFills()
              Spot price determines quantity.
              Engine searches for LPs that can fill that quantity.
              Pareto elimination → ranking → fill plan (max 5 legs).
              Reports fill plan to ORCH.
              If no valid plan: ORCH releases Gate 1 → FAILED.
        │
        └─▶ Reservation Engine — Gate 2
```

```
                Reserve qty from each LP vault in fill plan.
                Spot may have moved: quantity locked at this moment.
                LP internal balances decrease.
                If any leg fails: all Gate 2 releases + Gate 1 releases →
FAILED.
          │
          └─▶ ORCH writes DELIVERING to Ledger  ←── LIVE → DELIVERING
          │
          ├─▶ Execution Engine — beginDelivery()  (simultaneous)
          │     ORCH gives LP Tenet address(es).
          │     Single LP: co-sign LP vault → FI destination wallet.
          │     Multi-LP: co-sign each LP vault → Aggregation Vault → FI wallet.
          │     Reports: delivery confirmed + transaction hash.
          │
          └─▶ Settlement Engine — extractSettlement()  (simultaneous)
                ORCH gives FI Tenet address + amount + fee.
                Engine derives FI vault from Tenet address internally.
                Co-signs FI vault → Tenet Settlement Vault.
                Reports: extraction confirmed + transaction hash.
          │
          └─▶ ORCH waits for BOTH callbacks
                When both confirm:
          │
          └─▶ ORCH writes AWAITING SETTLEMENT  ←── DELIVERING → AWAITING
SETTLEMENT
          │
          └─▶ ORCH makes routing decision
                Same chain? → direct transfer
                Different chain, USDC? → CCTP instruction
                Different chain, not USDC? → Everclear instruction
                LP wants different asset? → Execution Engine finds swap LP
          │
          └─▶ Settlement Engine executes routing instruction
                CCTP: burn → attest → mint → confirm → report to ORCH
                Everclear: post intent → poll → confirm → report to ORCH
                Direct: transfer → confirm → report to ORCH
          │
          └─▶ ORCH gives Settlement Engine LP Tenet address(es) + amounts
                Single LP: one transfer.
                Multi-LP: proportional share per LP, one transfer each.
                Each LP receives payment on their native chain.
          │
          └─▶ Settlement Engine confirms all LP payouts → reports to ORCH
          │
          └─▶ ORCH releases Gate 2 LP reservations. LP internal balances
restore.
          │
          └─▶ ORCH writes COMPLETED to Ledger  ←── AWAITING SETTLEMENT →
COMPLETED

Done.
```

11. Integration & Wiring Points

The following marks every point in the architecture where external systems connect. These are the integration seams — the places where real custody infrastructure, real protocol APIs, and real price feeds plug in without changing any engine logic.

Wiring point	What connects here
VaultProvider.checkBalance()	Fireblocks GET /v1/vault/accounts/{id}/asset/{assetId}
VaultProvider.reserve()	Internal Tenet ledger operation — no external API. Enforced by co-sign policy.
VaultProvider.cosignTransfer()	Fireblocks POST /v1/transactions with TAP co-sign policy
Spot price feed	CryptoCompare API (primary) / CoinGecko (fallback). 30-second cache for CLI display. Real-time at quantity lock moment.
CCTP routing	Circle API: burn on source chain, poll for attestation, mint on destination chain.
Everclear routing	Everclear API: post transfer intent, poll for delivery confirmation.
FI vault portal	Registers FI workspace API key, vault account ID, Tenet co-signer key. Sets vault status from PENDING_CUSTODY to ACTIVE.
LP vault portal	Registers LP workspace API key per vault per chain. Sets vault status to ACTIVE. Adds vault to LP routing table.
Tenet admin portal	Registers Tenet's own Fireblocks workspace. Manages the Tenet Settlement Vault and Aggregation Vault lifecycle.
gRPC intent layer	Production transport between CLI and ORCH. Replaces the direct ORCH.submitIntent() function call with a typed Protocol Buffer message over gRPC. Engines do not change.

THIS IS THE ARCHITECTURE One intent. Four engines. One Ledger. Only ORCH writes. Engines are stateless. Vaults are internal balance sheets until execution. Funds never disappear. Every decision is logged. The architecture scales horizontally because nothing shares state except through the Ledger, and only one component touches the Ledger.

TENET SYSTEMS · Internal Architecture Whitepaper · v1.0 · Confidential
tenet-systems.netlify.app